

TemaFinale26 – Sprint 0: Analisi dei Requisiti

Requirements

Si riporta il **testo esatto del committente** (Protobook, cap. 31 – TemaFinale26):

"A Maritime Cargo shipping company (from now on, simply company) intends to automate the operations of load of containers in the ship's cargo hold (or simply hold). To this end, the company plans to employ a Differential Drive Robot (from now, called cargorobot). The hold is a rectangular, flat area with an Input/Output port (IOPort). The area provides 4 slots to store the containers and a slot named slot5."

- "• The slots1-4 depict the hold areas reserved to store one container each.*
- The slot5 depicts an area, where the cargorobot must temporarily store a container, before to place it in one of the slots1-4. During temporary storage, a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed.*
- The IOPort is a device with a pushbutton and a display. The pushbutton is pressed by the customer in order to send a request to load a container on the cargo. The display is used to show the answer to the request and to show the current state of the hold.*
- The sensor associated to the IOPort is a device (a sonar) used to detect the presence of a container, when it measures a distance D , such that $D < DFREE/2$, during a reasonable time (e.g. 3 secs)."*

TF2026 Requirements – testo esatto:

"The company asks us to build service named cargoservice that should work as follows. The cargoservice is able to receive a request to load a container sent by some customer by using the pushbutton of the IOPort.

- It sends the answer retrylater, if the IOPort is currently occupied by a container or if the system is Out of service.*
- It rejects the request when the hold is already full, i.e. the slots1-4 are already occupied.*
- Otherwise, it considers the system as engaged, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led. When the request to load is accepted, the customer must move the container in the sensor area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes disengaged. Then, the cargoservice uses the cargorobot to move the container from the IOPort to the slot5 (for marking the container) and then to the reserved slot. The service must also show on the display on the IOPort:*
 - the current state of the hold;*
 - the message 'Service working', when it is all is going well;*
 - the message 'Out of service' if the sonar sensor measures a distance $D > DFREE$ for at least 3 secs (perhaps a failure of the sonar)."*

Layout dell'area (dalla figura allegata al tema)

La figura del tema e' **illustrativa**: mostra le posizioni relative ma non formalizza l'area.

HOME	[SONAR]	HOME : posizione iniziale del cargorobot
[SLOT1][SLOT2]		SLOT1..4 : aree di stoccaggio container
robot	[SLOT5]	SLOT5 : area di marcatura temporanea
[SLOT3][SLOT4]		IOPORT: pushbutton + display + sensor
[IOPORT]		

Formalizzazione dell'area

Per ragionare in modo non ambiguo, l'area va **formalizzata** come una **mappa** a celle (griglia), in cui ogni cella ha coordinate (X,Y) e HOME, IOPort e slot1..slot5 occupano celle note. Una mossa del cargorobot porta da una cella a una cella adiacente.

Un **servizio DDR** tipicamente mette gia' a disposizione una **mappa built-in** dell'area: in tal caso le coordinate di HOME/IOPort/slot non sono un dato da inventare, ma provengono da quella mappa. (Vedi **D1**.)

Analisi dei requisiti

Goal: formalizzare le entita' del dominio escludendo le ambiguita' del linguaggio naturale (dichiarazioni Java per il dominio + modello qak per le interazioni).

Entita' formalizzate

Container — merce da caricare; nel TF26 non ha peso ne' PID a monte: riceve un **barcode** solo durante la marcatura in slot5.

Slot — area di stoccaggio della stiva (slot1..slot4 di carico; slot5 di marcatura):

```
public class Slot {
    String name;           // "slot1".."slot4" | "slot5"
    int    posX, posY;     // posizione nella mappa
    boolean occupied;
    boolean isMarkingArea; // true solo per slot5
}
```

Hold — aggregato degli slot di carico (slot1..4) + slot5 di marcatura:

```
public class Hold {
    Slot[] slots;           // slot1..slot4 (+ slot5 marcatura)
    boolean ioportOccupied(); // container presente all'IOPort
    boolean isFull();       // tutti gli slot1..4 occupati
    String reserveFreeSlot(); // nome di uno slot libero, "" se pieno
    void    confirmStored(String slotName);
    void    freeSlot(String slotName);
}
```

IOPort — dispositivo composto: **pushbutton** (input richiesta), **display** (output: risposta + stato hold) e un **sensor** (un sonar) per rilevare la presenza del container.

Sensor dell'IOPort (sonar) — misura la distanza D di un oggetto davanti all'IOPort. Condizioni fissate dal committente:

container presente : $D < D_{FREE}/2$ per ~ 3 s
sospetto guasto : $D > D_{FREE}$ per ≥ 3 s

Da chiarire col committente: cosa sia esattamente questo sensor e in che modo fornisca il dato (i requisiti non lo specificano). Va inoltre tenuto **distinto** dall'eventuale sonar montato sul **robot** (es. in WEnv): quest'ultimo non ha alcun ruolo nel cargoservice.

Marker — dispositivo in slot5 che etichetta il container con un barcode e **segnala** il completamento della marcatura.

LED — attuatore che **lampeggia** mentre il sistema e' *engaged*.

CargoRobot — il DDR fornito dal committente (risorsa data) che raggiunge una posizione-obiettivo su comando.

Assunzione da chiarire. Il committente/azienda dispone di *piu' software* per il DDR (un POJO come RobotObj26 e servizi a messaggi come RobotService26 o il pianificatore robotsmart): **quale** usare per realizzare il cargorobot e' una **decisione di progetto**, da motivare, non un requisito.

Struttura: componenti logici

- il **cargoservice** (il software da costruire);
- il **cargorobot** (il DDR fornito dal committente);
- i dispositivi dell'**IOPort**: pushbutton, display, sensor;
- il **marker** (in slot5) e il **LED**.

I componenti si scambiano informazioni. **Quanti** di essi siano realizzati come servizi autonomi, con **quale** tecnologia e **quale** trasporto, e' una decisione della fase di progetto da motivare: non e' un requisito.

Un modello (astratto) delle interazioni

A livello di requisiti interessa **quali** informazioni scorrono tra i componenti, non **come** vengano realizzate. Il cargoservice e' il componente centrale:

- riceve dal **pushbutton** una richiesta di carico e ne restituisce una risposta (slot riservato | retrylater | reject);

- aggiorna il **display** con lo stato corrente della stiva e i messaggi di servizio;
- e' informato dal **sensor** dell'IOPort della presenza del container e di un eventuale guasto;
- e' informato dal **marker** del completamento della marcatura;
- comanda il **cargorobot** a raggiungere una posizione;
- attiva il **LED** mentre il sistema e' *engaged*.

Assunzioni che NON vanno fissate nei requisiti (sono scelte della fase di progetto, da motivare):

- il **tipo** di interazione di ciascun flusso (evento, dispatch, request/reply): i requisiti non dicono, per esempio, se il sensor *emette eventi* o *invia dispatch*;
- il **trasporto** (MQTT, TCP, CoAP, ...): non e' un requisito che il sensor usi MQTT;
- la **tecnologia** di realizzazione (es. qak/QActor), il cui uso va motivato;
- quale **software del DDR** realizza il cargorobot.

La formalizzazione di questi flussi in un modello concreto e' compito dello Sprint successivo (Analisi del Problema) e del Progetto.

Core Business e Goal del primo prototipo

Domanda al committente: e' corretto concordare che il 'core business' consiste anzitutto nel **portare un container, accettata la richiesta, dall'IOPort allo slot riservato passando per la marcatura in slot5**, mentre la gestione del display, del LED lampeggiante e del 'Out of service' sono aspetti da consolidare in iterazioni successive?

Goal del primo prototipo (fase Development). Realizzare il **cargoservice** che, ricevuta una richiesta valida (pushbutton) con stiva non piena e IOPort libero, riserva uno slot, attende il container all'IOPort (entro il tempo previsto), comanda il cargorobot a portarlo **IOPort → slot5 (marcatura) → slot riservato**, e riporta il robot a HOME, tornando pronto per una nuova richiesta.

Nel primo prototipo si possono semplificare gli output (display/LED come log a video) e rinviare la gestione completa di *Out of service* e del barcode a un'iterazione successiva.

Domande al committente e chiarimenti ricevuti

D1. Posizioni di IOPort, HOME, slot1..slot5?

Chiarimento: un servizio DDR fornito ha gia' una **mappa built-in** dell'area; le coordinate provengono da quella mappa, non sono un dato da inventare.

D2. Come si rileva che l'IOPort e' "occupato da un container"?

(ancora da chiarire: e' lo stesso sensor della presenza, o un'informazione distinta?)

D3. Timeout 30s: cosa succede se scade?

Chiarimento: si assume che il container **NON sia arrivato**: il sistema si disengagia e libera lo slot riservato.

D4. Marcatura: il barcode va memorizzato?

Chiarimento: basta il **segnale** di completamento (markingDone); il barcode in se' non e' gestito dal cargoservice.

D5. Display: che cos'e' e cosa mostra?

Chiarimento: dovrebbe essere una **pagina web**, che **non** include il sensor, e deve mostrare almeno lo **stato corrente degli slot**.

D6. Out of service: le richieste in arrivo?

Chiarimento: vanno **rifiutate** (retrylater) finche' il servizio non torna attivo.

D7. Pushbutton: una sola richiesta per volta?

Chiarimento: si', come da requisito (le altre ricevono retrylater).

Problematiche aperte (per l'analista)

P1 — Realizzazione dei dispositivi senza hardware fisico.

Non disponendo del PicoW fisico, sensor, pushbutton, display, marker e LED vanno realizzati in modo **simulato** (in Python/Java). Cio' che conta a livello di requisiti e' l'**informazione** scambiata (per il sensor: la distanza D), non il dispositivo ne' il trasporto: sostituendo il simulato con l'hardware reale, le informazioni restano le stesse.

P2 — Natura e interazione del sensor dell'IOPort.

I requisiti non dicono **cosa** sia esattamente il sensor, ne' **come** comunichi il dato (evento? dispatch? quale trasporto?): sono decisioni di progetto da motivare. Il sensor va inoltre tenuto distinto dal sonar eventualmente montato sul **robot** (es. in WEnv), che **non riguarda** questa applicazione.

P3 — Movimento IOPort → slot5 → slot.

Il deposito e' modellato in modo astratto: il container si considera spostato quando il cargorobot **raggiunge** la posizione (slot5 e poi lo slot), tramite moverobot(X,Y), senza richiedere un attuatore di presa (non presente nel DDR).

P4 — Granularita' del Test Plan automatizzato (richiesto dalla valutazione).

Caso minimo significativo: una richiesta accettata che attraversa l'intero ciclo (pushbutton → slot riservato → container all'IOPort → IOPort→slot5→slot → HOME), verificando alla fine che lo stato della stiva sia coerente. Casi aggiuntivi: hold pieno (reject), IOPort occupato/out of service (retrylater), timeout (disengaged). **Questo Test Plan andrebbe formalizzato il prima possibile**, gia' a partire da questa fase.



By Alexander Kreuzer

Email: Alexander.kreuzer@studio.unibo.it

GIT repo: https://github.com/Alexing-uni/ISS26_Alexander_Kreuzer